

Introduzione e Architettura del Sistema

L'applicazione implementa una gestione distribuita di task basata sul metodo Kanban.

- **Lavagna (Server):** Agisce da coordinatore centrale, gestendo lo stato delle card (To Do, Doing, Done) e i dati degli utenti. È raggiungibile alla porta statica **5678**.
- **Utente (Client):** Partecipa al sistema registrandosi sulla Lavagna. Gli utenti utilizzano porte incrementali a partire dalla **5679**. Oltre a comunicare con il server, i client interagiscono direttamente tra loro per la fase obbligatoria di revisione dei task.

La comunicazione si basa interamente sul protocollo **TCP**, scelto per garantire l'affidabilità del recapito dei comandi (natura *loss-sensitive* dell'applicazione). Tutti i socket utilizzati sono TCP bloccanti.

Il formato dei messaggi dipende dal comando di riferimento, con i comandi utente che inviano tutti un singolo byte che rappresenta appunto il comando, mentre la lavagna ne invia 3 (a causa delle definizioni incrementali delle costanti associate ai comandi). Il formato dei messaggi viene definito prima delle comunicazioni e non viene mai inviata la lunghezza del buffer da leggere, tranne quando viene inviata la lista utenti che ha lunghezza variabile, questo per minimizzare le comunicazioni ed aumentare l'efficienza.

Il file definizioni.h contiene le definizioni di strutture dati e costanti.

Per compilare è sufficiente recarsi nella cartella progetto e usare:

```
gcc -Wall -o lavagna src/lavagna/lavagna.c
```

```
gcc -Wall -o utente src/utente/utente.c
```

Per avviare la lavagna: `./lavagna`, per avviare l'utente `./utente numero`, con numero ≥ 5679

Lavagna

Vengono inizializzate le strutture dati e creati i thread di ascolto degli input testuali e delle connessioni utente. Il primo tipo riconosce i comandi `SHOW_LAVAGNA` e `QUIT`, il secondo si mette in attesa del byte dell'utente che indica un comando tra:

- `HELLO`: gestisce la registrazione dell'utente e l'inserimento dei suoi dati nelle strutture apposite, se possibile gli assegna una card.
- `QUIT`: gestisce la disconnessione dell'utente e la riassegnazione di una eventuale card posseduta e non completata.
- `CREATE_CARD`: riceve id e testo della card da creare, controlla la disponibilità dell'id e inserisce la card nella lavagna, se possibile la assegna.
- `REQUEST_USER_LIST`: fa `SEND_USER_LIST`, cioè invia all'utente il numero di utenti connessi e successivamente la lista delle porte.
- `CARD_DONE`: sposta la card completata in done e se possibile ne assegna una nuova all'utente.

Gli assegnamenti delle card sono sempre gestiti da `HANDLE_CARD`, che si occupa anche di ricevere `ACK_CARD` e della creazione di un thread che si occuperà di fare `PING_USER` dopo 90 secondi dall'assegnazione. Gli spostamenti sono invece tutti gestiti da `MOVE_CARD`.

Dopo ogni modifica la lavagna viene ristampata con `SHOW_LAVAGNA`.

La concorrenza, limitata a letture e scritture sulla lavagna, è gestita attraverso il semaforo `mutex_lav`.

Utente

All'avvio avvengono i controlli sul numero di porta inserito, parte poi la connessione alla lavagna e successivamente la creazione di 2 thread, uno per gli input testuali e uno per le richieste ricevute dalla lavagna. Il thread originale rimane in attesa di connessioni p2p per le richieste di `REVIEW_CARD` degli altri utenti e che invia un `byte=1` come recensione positiva. Il thread che gestisce gli input accetta il comando `QUIT` che comporta la disconnessione, e `CREATE_CARD` che legge da tastiera l'id e il testo della nuova card e li invia alla lavagna dopo il `byte` che indica il comando. Il thread delle richieste lavagna invece è sempre in attesa di 3 `byte` che indicano un comando tra:

- **HANDLE_CARD**: riceve i dati della card, invia il `byte` di `ACK_CARD`, riceve la lista degli utenti e stampa la card a schermo. crea inoltre il thread dedicato alla richiesta delle revisioni (`REVIEW_CARD`) al termine della card (dopo 120 secondi dalla ricezione). dopo la ricezione di tutte le review invia `CARD_DONE` alla lavagna.
- **PING_USER**: fa `PONG_LAVAGNA`, inviando un `byte` come conferma.

La lista degli utenti è dichiarata come variabile globale e viene aggiornata dopo ogni ricezione di una nuova versione, la concorrenza su questa è gestita tramite il semaforo `mutex_u`.

Analisi Critica

Pregi:

- **Scalabilità**: L'uso di liste collegate per card e utenti permette al sistema di crescere dinamicamente senza vincoli prefissati di memoria, avendo come unico limite il numero massimo di porte per gli utenti e la dimensione degli interi per le card.
- **Affidabilità**: Il monitoraggio attivo tramite thread di ping garantisce che card assegnate a utenti crashati non rimangano bloccate indefinitamente in "Doing". Mentre l'assegnazione delle card alla connessione, disconnessione, creazione di una nuova card e al termine di un task svolto da parte degli utenti fa in modo che nessun utente o card rimanga bloccato in presenza di lavoro da svolgere.
- **Modularità**: La separazione tra gestione input e gestione richieste permette un'esperienza utente fluida.

Difetti:

- **Efficienza del Protocollo**: La conversione sistematica di interi in stringhe (es. porte e ID) aumenta leggermente il carico di calcolo e la dimensione dei pacchetti rispetto a un protocollo puramente binario.
- **Overhead P2P**: La fase di revisione richiede l'apertura di molteplici connessioni TCP temporanee verso tutti i peer, il che potrebbe diventare inefficiente in presenza di un numero molto elevato di utenti contemporanei.